Lab 5: Static Code Analysis Deliverables Report

Student Name: Preksha C M | **SRN:** PES1UG24CS685 | **Course:** Software Engineering (Lab 05)
Repository: L5_21_wildlife_sightings | **Target Script:** wildlife_sightings.py

---

1. Static Analysis Verification Summary

Static code analysis was executed on wildlife_sightings.py using Bandit (security vulnerabilities), Flake8 (PEP 8 grammar & formatting), and Pylint (code quality & logical anti-patterns). All issues were detected, documented, and resolved in three focused Git commits.

Analysis Tool	Focus Area	Before Fixes	After Fixes	Target Met?
Bandit	Security Vulnerabilities (CWE)	3 issues (1 High, 1 Med, 1 Low)	**0 issues** (Clean scan)	Yes (100% resolved)
Flake8	PEP 8 Formatting & Style	6 violations (E501, E741, E225)	**0 violations** (Clean exit)	Yes (All resolved)
Pylint	Code Quality & Anti-patterns	Score: **3.46 / 10**	Score: **10.00 / 10**	Yes (Exceeds > 9.5)
Runtime Output	Sanctuary Log Demonstration	5455c3 25 Leopard	5455c3 25 Leopard	Yes (Preserved)

2. Git Commit History (3 Commits Required by Handout)

Commit Hash	Commit Message	Tool Addressed
7001e3e	fix(security): resolve Bandit vulnerabilities (B608 SQL injection, B324 MD5 hashing, B101 assert usage)	Bandit (Security)
5fbd9f6	style: fix PEP 8 issues detected by Flake8 (E501 line lengths, E741 ambiguous name, E225 whitespace)	Flake8 (Style/PEP 8)
9567a5b	refactor(quality): resolve Pylint issues (10/10 score), add reports, issues table, and reflection	Pylint (Quality & Docs)

3. Documented Issues and Fix Approaches Table

Issue	Type	Line(s)	Description	Fix Approach
B324: Weak MD5 Hash	Security (High)	15	Use of broken MD5 hash algorithm detected.	Added usedforsecurity=False for non-cryptographic short observer ID.
B608: SQL Injection Vector	Security (Med)	9	String formatting (.format) in INSERT query exposes database to injection.	Converted query to use parameterized placeholders (?) with tuple args.
W0102: Mutable Default Arg	Bug/Logic (Med)	8	extras={} binds once at def time, sharing mutated dict across calls.	Defaulted to extras=None and dynamically initialized inside function.
B101: Assert Used in Code	Security/Reliability (Low)	36	assert len(totals) == 3 is removed under Python optimization (-O).	Replaced with explicit conditional check raising ValueError.
W0621: Redefining Outer Name	Code Quality (Warning)	8, 18, 26	Global conn shadowed function parameter names.	Removed global conn and wrapped DB creation in init_db().
E741: Ambiguous Name	Style (PEP 8)	28	Variable l is ambiguous and easily mistaken for 1 or I.	Renamed variable to descriptive identifier rarest_row.
E225: Operator Spacing	Style (PEP 8)	28	l=min(...) missing spaces around assignment operator.	Added standard single whitespace around =.

E501 / C0301: Line Length	Style (PEP 8)	5, 9, 19, 27	Multiple SQL strings and calls exceeded 79 and 100 character limits.	Split long SQL queries across multiple lines cleanly.
C0114/C0116: Missing Docs	Convention	Module & all funcs	Missing module and function docstrings explaining purpose, args, and returns.	Added complete PEP 257 docstrings for module and all 5 functions.
C0103: Naming Convention	Convention	8, 14, 18	camelCase names (logSighting, etc.) violate Python snake_case.	Converted to snake_case and added backward-compatible legacy aliases.
R1705: Unnecessary Else	Refactor	20-23	Redundant else branch immediately after a return statement.	Removed else and de-indented terminal return.

4. Reflection Questions and Answers

Q1: What are the distinct roles and scopes of Pylint, Bandit, and Flake8, and why is it essential to use all three tools in a modern Python development pipeline?
Answer:
• **Pylint** performs deep semantic AST analysis to evaluate software design, maintainability, variable shadowing, code duplication, docstrings, and logical antipatterns like mutable default arguments (W0102). It provides an overall numerical quality score.
• **Bandit** inspects code specifically for security vulnerabilities and weaknesses (CWEs) such as injection flaws, hardcoded credentials, and cryptographically insecure functions.
• **Flake8** combines PyFlakes, Pycodestyle, and McCabe complexity to strictly enforce PEP 8 style, line lengths, spacing, and syntactic clarity.
Combining all three tools delivers *defense-in-depth*: Flake8 guarantees readability and uniformity across engineering teams, Pylint prevents subtle logic errors and technical debt, and Bandit safeguards against vulnerabilities prior to deployment.

Q2: What are the security risks associated with SQL injection (B608), weak hashing algorithms (B324), and assert statements (B101), and how did the implemented fixes resolve them?
Answer:
• **SQL Injection (B608):** String interpolation (e.g., .format()) permits untrusted inputs to modify database command structure. The fix utilizes parameterized query placeholders (?), ensuring values are escaped and passed safely as query parameters.
• **Weak Hash Usage (B324):** MD5 is cryptographically compromised and susceptible to collision exploits. In non-cryptographic contexts (generating short visual observer hashes), passing usedforsecurity=False explicitly documents the non-security intent to the runtime and static analyzers.
• **Assert Statements (B101):** Python compiler strips assert statements entirely when running in optimized bytecode mode (python -O). Critical integrity checks implemented via assert are bypassed in production. The fix replaces assertions with explicit if ...: raise ValueError(...) defensive checks.

Q3: Why is using a mutable default argument (e.g., extras={}) considered a dangerous anti-pattern in Python, and how does extras=None prevent unintended side-effects?
Answer:
Python evaluates and binds default parameter expressions exactly once when the function definition is executed at module load time. Consequently, when a mutable structure like a dictionary (extras={}) is used as a default value, all function calls that omit the argument mutate and accumulate data in the identical dictionary instance in memory. This introduces severe cross-call side-effects, corrupted records, and hidden race conditions. Setting extras=None and initializing extras = {} inside the function scope creates a fresh, isolated dictionary instance for each individual invocation.

Q4: How does integrating automated static code analysis into continuous integration (CI/CD) pipelines impact overall software maintainability and team productivity?
Answer:
• **Shift-Left Defect Detection:** Discovering defects, security issues, and style violations at the commit stage drastically decreases the remediation cost compared to late-stage QA or production incidents.
• **High-Value Code Reviews:** Human reviewers spend time verifying domain logic, system architecture, and algorithmic efficiency rather than nitpicking formatting conventions.
• **Automated Quality Gates:** Enforcing thresholds (e.g., zero Bandit findings, Flake8 clean, Pylint score > 9.5) reliably prevents gradual degradation and technical debt accumulation across pull requests.

5. Clean Refactored Script (wildlife_sightings.py - 10.00/10)

```
"""Wildlife Sanctuary Sightings Log.

This module provides functionality to record wildlife sightings,
generate observer identifiers, summarise sighting counts by zone,
and determine the rarest recorded species.
"""

import hashlib
```

```
import sqlite3


def init_db():
    """Initialize in-memory SQLite database connection with sightings table."""
    connection = sqlite3.connect(":memory:")
    connection.execute(
        "CREATE TABLE sightings ("
        "id INTEGER PRIMARY KEY, species TEXT, zone TEXT, count INTEGER)"
    )
    return connection


def log_sighting(conn, species, zone, count, extras=None):
    """Record a sighting and update species count dictionary.

    Args:
        conn: SQLite database connection.
        species: Name of the animal species.
        zone: Geographic zone where animal was sighted.
        count: Number of individuals sighted.
        extras: Optional dictionary mapping species to total counts.

    Returns:
        dict: Updated dictionary of species counts.
    """
    if extras is None:
        extras = {}
    conn.execute(
        "INSERT INTO sightings (species, zone, count) VALUES (?, ?, ?)",
        (species, zone, count)
    )
    extras[species] = extras.get(species, 0) + count
    return extras


def observer_id(observer):
    """Generate a short hexadecimal identifier for an observer.

    Args:
        observer: Observer's name or string identifier.

    Returns:
        str: Truncated hexadecimal MD5 digest (6 chars).
    """
    return hashlib.md5(
        observer.encode(), usedforsecurity=False
    ).hexdigest()[:6]


def zone_summary(conn, zone):
    """Calculate total sightings count for a specific zone.

    Args:
        conn: SQLite database connection.
        zone: Geographic zone to summarize.

    Returns:
        int: Total sightings count in the specified zone.
    """
    query = "SELECT SUM(count) FROM sightings WHERE zone = ?"
    row = conn.execute(query, (zone,)).fetchone()
    if row[0] is None:
        return 0
    return row[0]


def rarest(conn):
    """Find the species with the minimum total sightings count.

    Args:
        conn: SQLite database connection.
    Returns:
```

```
        str: Name of the rarest recorded species.
    """
    query = "SELECT species, SUM(count) FROM sightings GROUP BY species"
    rows = conn.execute(query).fetchall()
    rarest_row = min(rows, key=lambda r: r[1])
    return rarest_row[0]


# Backward-compatible aliases for original camelCase function names
logSighting = log_sighting  # pylint: disable=invalid-name
observerId = observer_id    # pylint: disable=invalid-name
zoneSummary = zone_summary  # pylint: disable=invalid-name


def main():
    """Execute the demonstration sightings workflow."""
    conn = init_db()
    totals = log_sighting(conn, "Leopard", "North", 1)
    log_sighting(conn, "Chital", "North", 24, totals)
    log_sighting(conn, "Hornbill", "East", 3, totals)
    if len(totals) != 3:
        raise ValueError("Expected 3 species recorded in totals")
    print(observer_id("Arun"), zone_summary(conn, "North"), rarest(conn))


if __name__ == "__main__":
    main()
```